Final CADP 🚀

Se aprueba o se aprueba.

Resumen Teórico

Programa = Algoritmo + Datos

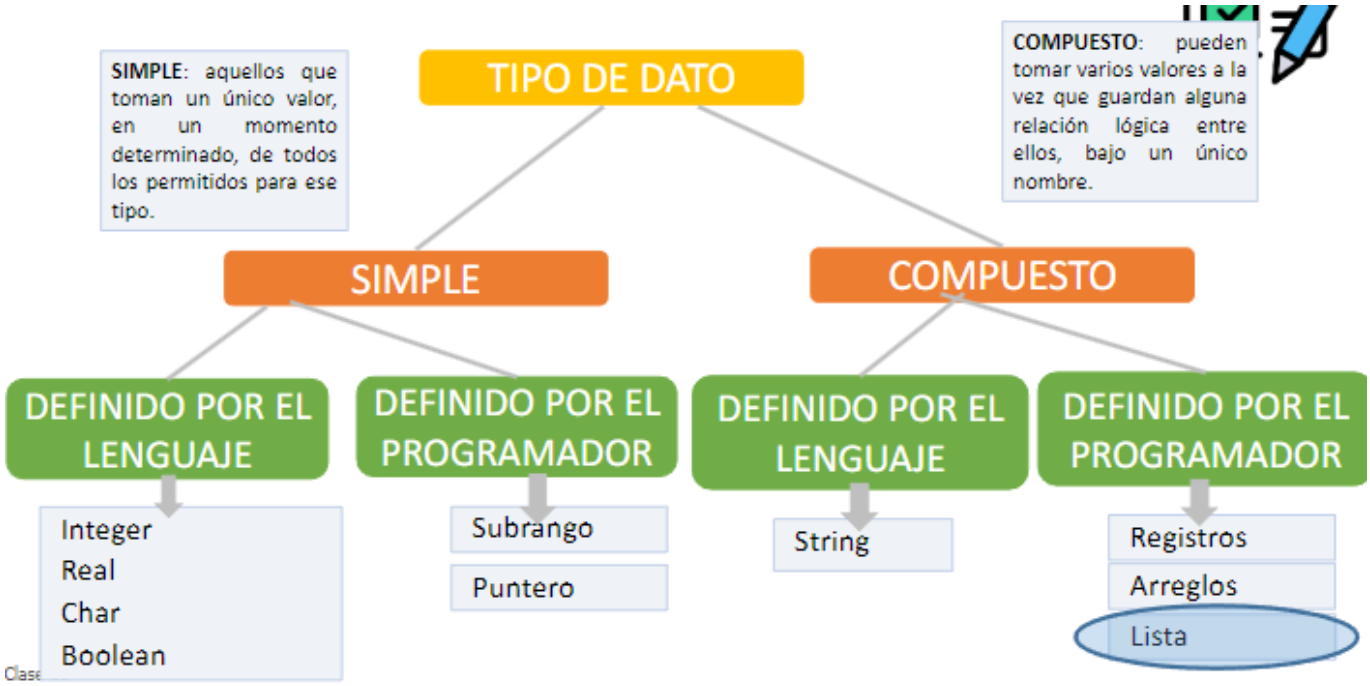
Algoritmo:

Secuencia de pasos (instrucciones) a realizar sobre una computadora para alcanzar un resultado deseado.

Datos:

Es una representación de un objeto del mundo real mediante el cual podemos modelizar aspectos del problema sobre una computadora.

Tipos de datos:



- **Simples**

  Son aquellos que no están compuestos por otros tipos de datos, sino que representan valores atómicos. Los tipos de datos simples en Pascal incluyen:

  - Tipos ordinales (tienen un orden natural, como números enteros):

    - `Integer`: Números enteros.
    - `Byte`: Enteros sin signo entre 0 y 255.
    - `ShortInt`: Enteros cortos entre -128 y 127.
    - `LongInt`: Enteros largos.
    - `Word`: Enteros sin signo entre 0 y 65535.

        - `Boolean`: Valores lógicos (`True` o `False`).
        - `Char`: Caracteres individuales.
        - `Enumerated`: Tipo definido por el usuario con valores en secuencia, por ejemplo:

          ```
          type Days = (Sun, Mon, Tue, Wed, Thu, Fri, Sat);
          ```

        - `Subrange`: Un rango de un tipo ordinal, por ejemplo: `1..10` o `'A'..'Z'`.

    - Tipos numéricos reales:

        - `Real`: Números en punto flotante.
        - `Single`, `Double`, `Extended`: Variantes del tipo real que representan números de punto flotante con diferentes precisiones.

    - **Definido por el programador**: Permiten definir nuevos tipos de datos a partir de los tipos simples.

- **Compuestos**

  Pueden tomar varios valores a la vez que guardan alguna relación lógica entre ellos, bajo un único nombre. Los tipos de datos compuestos en Pascal incluyen:

    - Arreglos:

      ```
      var numbers: array[1..5] of Integer;
      ```

    - Registros:

      ```
      type
        Person = record
          name: string;
          age: Integer;
        end;
      var
      ```

Tipos de variables:

- **Variables**

  Es una zona de memoria cuyo contenido va a ser alguno de los tipos mencionados anteriormente. La dirección inicial de esta zona se asocia con el nombre de la variable. Puede cambiar su valor durante el programa.

- **Constante**

Es una zona de memoria cuyo contenido va a ser alguno de los tipos mencionados anteriormente. La dirección inicial de esta zona se asocia con el nombre de la constante. NO puede cambiar su valor durante el programa.

Estructuras de control

- **Decisión**

```
if condición then
  instrucción1
else
  instrucción2;
```

Datos importantes ❗

  - Dentro del cuerpo del if puede ir cualquier cosa.
  - En la condicion se puede utilizar:
    - Comparaciones que devuelvan True o False.
    - Funciones que devuelvan un valor booleano.
    - Expresiones lógicas combinadas.
    - Verificaciones de pertenencia a conjuntos.
    - Variables booleanas directamente.

- **Iteracion Precondicional**

```
while condición do
    instrucción;
```

Datos importantes ❗

  - Evalua y ejectura en caso de ser verdadera la condicion
  - En la condicion sucede lo mismo que en el if
  - Puede ejecutarse 0, 1 o más veces.

- **Iteracion Postcondicional**

```
repeat
    instrucción;
until(condición);
```

Datos importantes ❗

  - Ejecutra y luego evalua
  - En la condicion sucede lo mismo que en el if
  - Se repite mientras la condición es falsa.

  - Siempre se ejecutara una vez la instruccion(Se puede evitar con un if antes pero caemos en que termina siendo un while).
  - Puede ejecutarse 1 o más veces.

- **Repeticion**

```
for i:=1 to 10 do begin
    instruccion;
end;
```

Datos importantes ❗

  - La variable índice debe ser de tipo ordinal
  - La variable índice no puede modificarse dentro del lazo
  - La variable índice se incrementa y decrementa automáticamente
  - Cuando el for termina la variable índice no tiene valor definido

- **Case**

```
case (variable) of
    condicion1: accion1;
    condicion2: accion2;
end;
```

Ejemplo

```
case (car) of
    car = ‘a’: cmin:= cmin + 1;
    car = ‘A’: cmay:= cmay + 1;
end;
```

Datos importantes ❗

  - La variable índice debe ser de tipo ordinal
  - Las opciones deben ser disjuntas. Cada valor debe ser único y no puede solaparse con otros valores. si pones dos opciones en las que puede entrar no compila.

Modularizacion

- Cada subproblema está en un mismo nivel de detalle.
- Cada subproblema puede resolverse independientemente.
- Las soluciones de los subproblemas puede combinarse para resolver el problema original.

Ventajas

- Mayor Productividad

- Reusabilidad
- Facilidad de crecimiento
- Legibilidad

Procedimientos

Un procedimiento en Pascal es un bloque de código que realiza una tarea específica. A diferencia de una función, un procedimiento no devuelve un valor al llamado:

```
procedure Saludar(nombre: string);
begin
    WriteLn('Hola, ', nombre, '!');
end;
```

Funcion

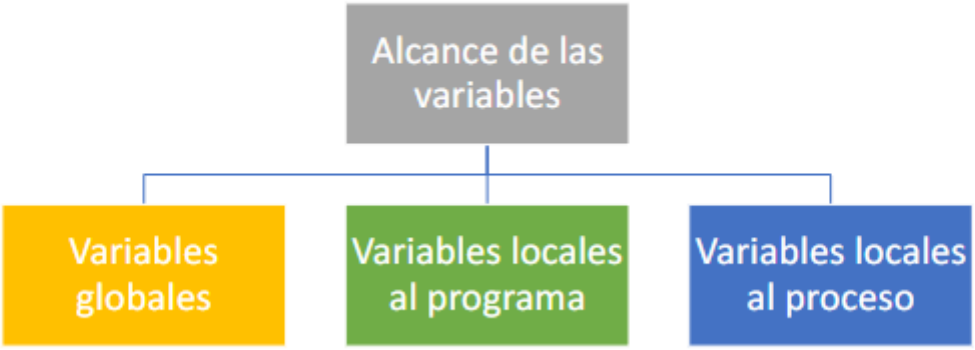
Conjunto de instrucciones que realizan una tarea especifica y retorna un único valor de tipo simple.
(Definicion en teorias) Realmente puede retornar un tipo registro que es un tipo de dato compuesto.(Ver línea 16.)
Codigo a modo de muestra:

```
program ejer;
type
    TPersona = record
        Nombre: string;
        Edad: Integer;
    end;

function CrearPersona(Nombre: string; Edad: Integer): TPersona;
begin
    CrearPersona.Nombre := Nombre;
    CrearPersona.Edad := Edad;
end;

var
    persona: TPersona;
begin
    persona := CrearPersona('Juan', 30);
    WriteLn('Nombre: ', persona.Nombre);
    WriteLn('Edad: ', persona.Edad);
end.
```

Alcance de variables



Casos

- Si es una variable utilizada en un proceso
  - 1° Se busca si es variable loca
  - 2° Se busca si es un parámetro
  - 3° Se busca si es variable global al programa
- Si es una variable usada en un programa
  - 1° Se busca si es variable local al programa
  - 2° Se busca si es variable global al programa

Comunicacion entre modulos

- Variables globales:
  - Demasiados identificadores
  - No se especifica la comunicación entre los módulos
  - Conflictos de nombres de identificadores utilizados por diferentes programadores.
  - Posibilidad de perder integridad de los datos.
- Parametros:
  - El uso de parámetros significa que los datos compartidos se deben especificar como parámetros que se trasmiten entre módulos.
  - **Parámetros por valor**: Con él puede realizar operaciones y/o cálculos, pero no producirá ningún cambio ni tampoco tendrá incidencia fuera del módulo.

```
procedure uno (num: integer);
```

  - **Parametros por referencia**: Puede operar con ella y su valor original dentro del módulo, y las modificaciones que se produzcan se reflejan en los demás módulos que conocen la variable.

```
procedure uno (var num: integer);
```

- El número y tipo de los argumentos utilizados en la invocación a un módulo deben coincidir con el número y tipo de parámetros del encabezamiento del módulo.

- Un parámetro por valor debiera ser tratado como una variable de la cuál el módulo hace una copia y la utiliza localmente. Algunos lenguajes permiten la modificación local de un parámetro por valor, pero toda modificación realizada queda en el módulo en el cual el parámetro es utilizado.
- El número y tipo de los argumentos utilizados en la invocación a un módulo deben coincidir con el número y tipo de parámetros del encabezamiento del módulo.

ESTRUCTURA DE DATOS

Permite al programador definir un tipo al que se asocian diferentes datos que tienen valores lógicamente relacionados y asociados bajo un nombre único.

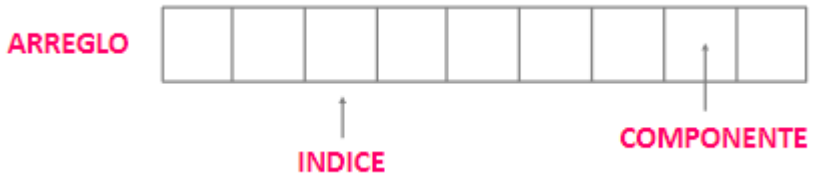
Clasificacion

- Por Elementos
  - Homogenea: Los elementos que la componen son del mismo tipo
  - Heterogenea: Los elementos que la componen pueden ser de distinto tipo
- Por Acceso
  - Secuencial: Para acceder a un elemento particular se debe respetar un orden predeterminado, por ejemplo, pasando por todos los elementos que le preceden, por ese orden.
  - Directo: Se puede acceder a un elemento particular, directamente, sin necesidad de pasar por los anteriores a él, por ejemplo, referenciando una posición.
- Por Tamaño
  - Estatica: EL tramaño dela estructrua no varia durante la ejecucion del programa
  - Dinamica: El tamaño de la estructura puede variar durante la ejecucion del programa
- Por linealidad
  - Lineal: Esta formada por ningun, un o varios elementos que guardan una relacion adyacente ordenada donde a cada elemento le sigue uno y le precede uno solamente.
  - No lineal: Para un elemento dado puede existir 0,1 o mas elementos que le seuceden y 0,1 o mas elementos que le preceden.

Tipo de datos

Arreglos

Un **arreglo** (ARRAY) es una estructura de datos compuesta que permite acceder a cada componente por una variable índice, que da la posición de la componente dentro de la estructura de datos.



Vectores

Es una colección de elementos que se guardan consecutivamente en la memoria y se pueden referenciar a través de un índice.

- HOMOGENEA Los elementos pueden son del mismo tipo

- ESTATICA El tamaño no cambia durante la ejecución (se calcula en el momento de compilación)
- INDEXADA Para acceder a cada elemento de la estructura se debe utilizar una variable 'índice' que es de tipo ordinal. **Datos importantes ❗**
- Permite rapido acceso por tener indice(Que tiene que ser de tipo ordnial)
- Se utiliza en casos en los que se sabe la longitud, y se precisa un acceso rapido
- No se utiliza cuando no se conoce la longitud y se precisa agregar y eliminar de manera mas eficiente.

```
Program uno;
Type
vector = array [rango] of tipo;
```

- Dimensiones
    - DIMENSION FISICA Se especifica en el momento de la declaración y determina su ocupación máxima de memoria. La cantidad de memoria total reservada no variará durante la ejecución del programa.
    - DIMENSION LOGICA Se determina cuando se cargan contenidos a los elementos del arreglo. Indica la cantidad de posiciones de memoria ocupadas con contenido real. Nunca puede superar la dimensión física.

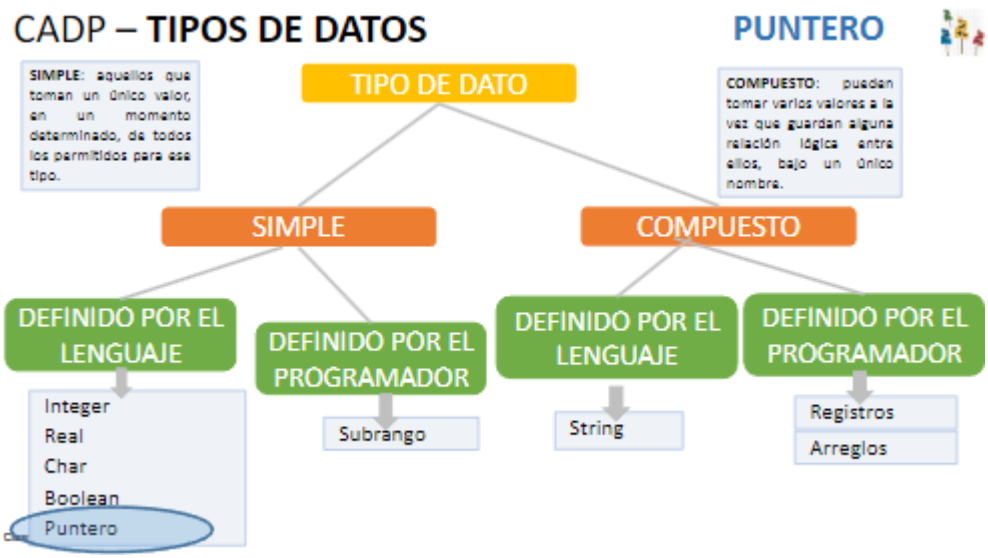
Operaciones vistas en la teoria

- Carga de valores
- Lectura / Escritura
- Recorridos
- Agregar elementos al final
- Insertar elementos (Corrimientos)
- Borrar elementos (Corrimientos)
- Búsqueda de un elemento (Busqueda mejorada O(n), Busqueda dicotomica O(log n))
- Ordenación de los elementos

Alocacion estatica y alocacion dinamica

- Variables estaticas No permiten modificar su tamaño en timepo de ejecucion. Las variables y tipos reservan memoria en su declracion y se mantienen durante todo el programa. El lenguaje peude validar previo a la ejecucion.

- Variables dinamicas Permiten modificar en timep ode ejecucion la memoria utilizada



Puntero

Es un tipo de variable usada para almacenar una direccion en memoria. En esa diraccion de memoria se encuentra el valor que puede ser de cualquiera de los tipos vistos. Un puntero es un **tipo de dato simple**.

Declaracion:

```
Program tres;
Type
puntero = ^integer;
```

Operaciones

- Creacion de una varaible puntero (new (puntero))
- Liberacion (puntero:=nil)
  - Libera la conexión que existe entre la variable y la posición de memoria.
  - La memoria sigue ocupada.
  - La memoria no se puede referenciar ni utilizar.
- Destruccion dispose (puntero)
  - Libera la conexión que existe entre la variable y la posición de memoria.
  - Libera la posición de memoria.
  - La memoria liberada puede utilizarse en otro momento del programa.
- Asignacion entre variables puntero (puntero1 := puntero2)
- Asignacion de un valor al contenido de una variable puntero (puntero^:= dato)
- Comparacion de una vairable puntero (puntero1 = puntero2)

Datos importantes ❗

- if (p = nil) then, compara si el puntero p no tiene direccion asignada
- if (p = q) then, compara si los punteros p y q apuntan a la misma direccion de memoria
- if (p^ = q^) then,compara si los puntero p y q tienen el mismo contenido.
- no se puede hacer read(p), si p es una variable de tipo puntero
- no se puede hacer write(p) si p es una variabel de tipo puntero

- no se puede asignar una direccion a un puntero de manera manual p:=ABCD
- no se puede comprar por mayor o menor direcciones de punteros (p>q)

Calculo de memoria

Memoria de un programa: En rasgos generales la memoria necesaria para la ejecucion de un programa puede dividirse en dos:

- **Memoria Estatica**: variables locales y globales del programa.
- **Memoria Dinamica**: Solo cuando en la ejecucion de un programa se reserva o libera memoria.

Listas

Coleccion de nodos, donde cada nodo contiene un elemento y en que direccion de memotia se enceuntra el siguiente nodo.

Declaracion:

```
Program uno;
Type
  lista = ^nodo;
  nodo = record
    dato: integer;
    sig: lista;
  end;
Var
  pri: lista; {Memoria estática reservada}
```

Operaciones

- Creación de una lista.
- Agregar nodos al comienzo de la lista.
- Recorrido de una lista.
- Agregar nodos al final de la lista.
- Insertar nodos en una lista ordenada
- Eliminar nodos de una lista

Correccion y eficiencia

Correccion

Un programa es correcto si se realiza de acuerdo a sus especificaciones

Tecnicas

- Testing: Consiste en proveer evidencias convincentes que el programa hace el trabajo esperado -En esta tecnica aparece el plande de pruebas, que consiste en decidicir que ver que aspectos del programa deben ser testeados y econtrar datos de pruebas de cada uno de esos aspectos.

- Debugging: Es el proceso de descubrir y reparar la causa del error. Es agregar setencias adicionales en el programa para poder manitorear su comportamiento mas cercanamente.
- Walkthrough: Es recorrer un programa frente a una audiencia.
- Verificacion: Significa controlar las pre y pos condiciones del mismo.

Eficiencia

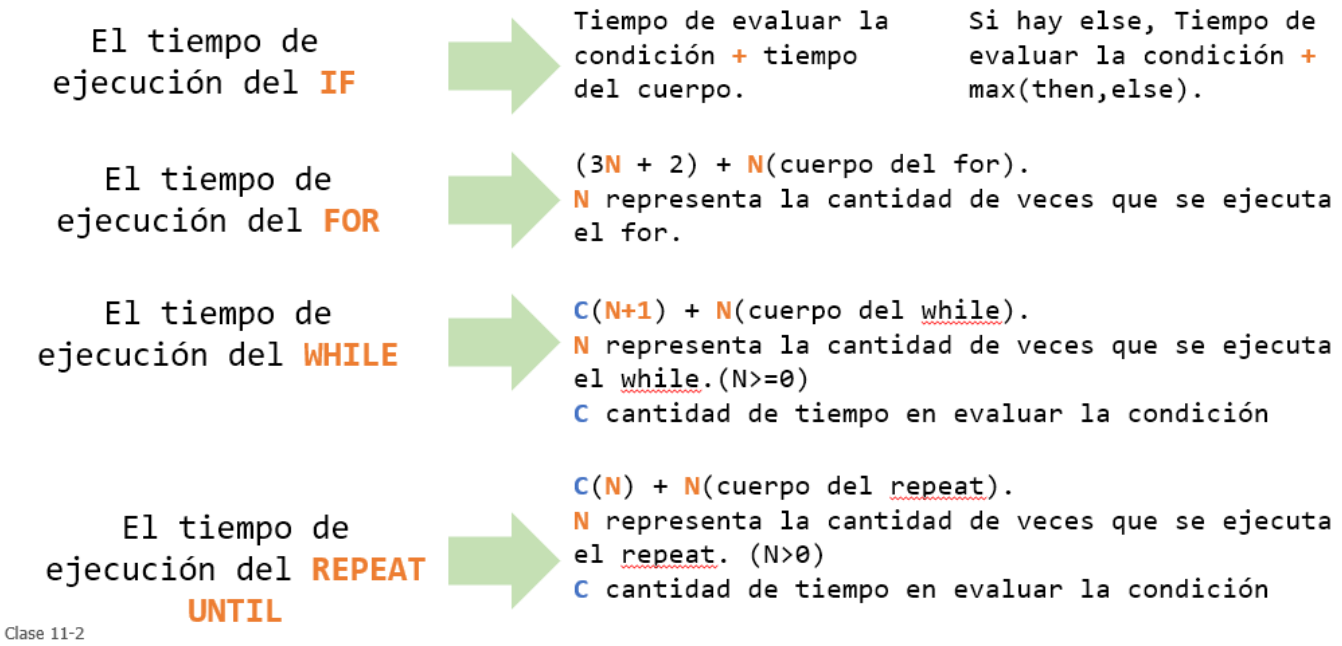
El analisis de la eficiencia de un algoritmo estudia el timepo que tarda un algoritmo en ejecutarse y la memoria que requiere.

Tiempos de ejecucion

- Analisis empirico: Es necesario realizar el programa y medir el tiempo consumido.
- Analisis teorico: Imploica encontrar una cota maxima para expresar el tiempo de nuestro algoritmo, sin necesidad de ejecutarlo.

Calcuclo de timepos de ejecucion (Analisis teorico)

Dado un algoritmo que es correcto se calcula el timepo de ejecucion de cada una de sus intrucciones. Solo las intrucciones elementales del algoritmo: Asignacion y operaciones aritmetico/logicas Una instruccion elemental utiliza un tiempo contrante para su ejecucion. 1UT.



- Estructura IF evaluar condicion (c/u 1UT) + cuerpo del if. Se suma el peor de los casos if o else.

- Estructura FOR (3N + 2) + N(Cuerpo) Siendo N la cantidad de veces que se ejecuta el for

Ejemplo:

ANALISIS TEORICO

T (alg) = T(1) + T(2) + T(3)

```
Program uno;
var
 i,temp,x: integer;

Begin
 (1) aux:= 8;
 (2) for i:= 4 to 9 do
       begin
        x:= aux;
        aux:= aux + 5;
       end

 (3) aux:= aux + 1;
end;
```

T(1)= asignación 1 UT.

T(2)= for= 3N+2 + N(cuerpo)

N= 6 (LS-LI + 1)
(3*6) +2 + 6(cuerpo)
20 + 6(cuerpo)
cuerpo = 1 + 2 = 3UT

>> 20 + 6(3) =

= 20 + 18 = 38UT

T(3)= 1 + 1 = 2 UT.

T (alg) = 41 UT

Clase 11-2

- Estructura WHILE C(N+1)+ N(cuerpo) Siendo C la condiciones
- Estructura REPEAT C(N) + N(cuerpo)

Ejemplo:

ANALISIS TEORICO

T (alg) = T(1) + T(2) + T(3)

```
Program uno;
var
 i,temp,x: integer;

Begin
 (1) aux:=0;
 (2) while (aux >= 0) and (aux<5) do
       begin
        x:= aux;
        aux:= aux + 1;
       end

 (3) aux:= aux + 1;
end;
```

T(1)= 1 UT.

T(2)= while= C(N+1)+ N(cuerpo)

C= (1) + (1) + conector = 3
N = 5
3*(5+1) + 5(cuerpo)
cuerpo = 1 + 2 = 3UT

>> 18 + 5(3) =

= N + 18 + 15 = 33 UT

T(3)= 1 + 1 = 2 UT.

T (alg) = 1 + 33 + 2 = 36 UT

Clase 11-2

- Estructura REPEAT C(N) + N(cuerpo)

ANALISIS TEORICO

T (alg) = T(1) + T(2) + T(3)

```
Program uno;
var
 i,temp,x: integer;

Begin
 (1) aux:=0;
 (2) repeat
       x:= aux;
       aux:= aux + 1;
     until (aux > 5)

 (3) aux:= aux + 1;
end;
```

T(1)= 1 UT.

T(2)= repeat= C(N)+ N(cuerpo)

C= 1
N = 5
1*(5) + 5(cuerpo)
cuerpo = 1 + 2 = 3UT

>> 5 + 5(3) =

= N + 5 + 15 = 20 UT

T(3)= 1 + 1 = 2 UT.

Clase 11-2

T (alg) = 1 + 20 + 2 = 23 UT